SECTION A – DEBUGGING QUESTIONS

Question 1: Operator Precedence Trap with Bitwise Operations & Implicit Promotion

1. Identify the operator precedence conflict on Line 7.

The expression is:

if (num & 1 == 1 && num > 10)

The equality operator (==) has higher precedence than the bitwise AND operator (&). Therefore, the compiler evaluates the expression as:

num & (1 == 1) && num > 10

Since (1 == 1) evaluates to 1, the expression becomes:

(num & 1) && (num > 10)

This may produce logical errors because the programmer intended to check whether the number is odd. Parentheses must be used to ensure correct evaluation.

Correct expression:

if ((num & 1) == 1 && num > 10)

2. Why does removing the break statement result in an infinite loop?

The loop is:

for (unsigned char i = 3; i >= 0; --i)

An unsigned char can store only non-negative values. When i reaches 0 and is decremented, it wraps around to 255 instead of becoming -1.

Therefore, the condition i >= 0 always remains true, resulting in an infinite loop.

Using an integer variable instead of an unsigned char prevents this problem.

3. Corrected C++ Code

```
#include <iostream>
using namespace std;
```

```
int main() {
    unsigned char num = 15;

    if ((num & 1) == 1 && num > 10) {
        cout << "Valid Odd Number: " << (int)num << endl;
    } else {
        cout << "Condition Failed for: " << (int)num << endl;
    }

    for (int i = 3; i >= 0; --i) {
        cout << i << " ";
    }

    cout << endl;

    return 0;
}
```

4. Dry Run and Expected Output

Step	num	Condition	Result
1	15	(15 & 1) == 1	True
2	15	15 > 10	True
3	15	Both conditions true	Valid Odd Number
4	15	Loop starts at 3	Prints 3 2 1 0

Output:

```
Valid Odd Number: 15
3 2 1 0
```

Question 2: Scope Resolution & Variable Shadowing Conflicts

1. Identify the syntax error on Line 11.

The incorrect statement is:

double totalWithGlobalTax = itemPrice + (itemPrice * :taxRate);

The scope resolution operator is incorrectly written as :taxRate.

The correct operator consists of two colons (::) and is used to access a global variable.

Correct statement:

double totalWithGlobalTax = itemPrice + (itemPrice * ::taxRate);

2. Explain what value ::taxRate resolves to on Line 16.

The expression ::taxRate refers to the global variable taxRate, whose value is 0.18.

The unary scope resolution operator cannot directly access an enclosing local function variable. It accesses the global namespace.

When variables have the same name in different scopes, the inner variable shadows the outer variable.

Therefore:

- Global taxRate = 0.18 (18%).
- Local taxRate = 0.05 (5%).
- Inner taxRate = 12 (integer).

3. Corrected C++ Code

```
#include <iostream>
using namespace std;

double taxRate = 0.18;

int main() {
   double itemPrice = 1000.0;
   double localTaxRate = 0.05;

   cout << "Local Rate: " << localTaxRate << endl;

   double totalWithGlobalTax =
      itemPrice + (itemPrice * ::taxRate);

   {
      int corporateTaxRate = 12;
```

```
        double totalWithLocalTax =
            itemPrice + (itemPrice * localTaxRate);

        cout << "Inner total: " << totalWithLocalTax << endl;
    }

    cout << "Final Global Calculation: "
         << totalWithGlobalTax << endl;

    return 0;
}
```

4. Expected Console Output

Local Rate: 0.05
Inner total: 1050
Final Global Calculation: 1180

Explanation:

- Local calculation = 1000 + (1000 × 0.05) = 1050.
- Global calculation = 1000 + (1000 × 0.18) = 1180.

Question 3: Switch-Case Variable Initialization & Illegal Case Types

1. Why does initializing int multiplier = 5 inside case 2 cause a compiler error?

In a switch statement, control can jump directly to different case labels. Declaring and initializing a variable directly inside a case without creating a separate block can cause a cross-initialization error.

The compiler may allow execution to jump over the initialization of a variable, which is not permitted in this situation.

The solution is to enclose each case containing variable declarations inside curly braces { }.

2. Why is case 4.5 rejected?

The switch statement in C++ supports integral types such as:

- int
- char

- bool
- enum

Floating-point values such as float and double are not permitted as switch expressions or case labels.

Therefore, case 4.5: is invalid. It should be replaced with an appropriate integer case value.

3. Identify the logical defect due to the missing break statement.

The break statement is missing after case 2.

As a result, when tokenType == 2, execution continues into case 3. This is called fall-through.

It causes an additional 100 to be added to the calculated value, producing an incorrect result.

4. Corrected C++ Code

```
#include <iostream>
using namespace std;

int main() {
   int tokenType = 2;
   float tokenValue = 0.0f;

   switch (tokenType) {
      case 1: {
         int offset = 10;
         tokenValue = 50.0f + offset;
         break;
      }

      case 2: {
         int multiplier = 5;
         tokenValue = 20.0f * multiplier;
         break;
      }

      case 3: {
         tokenValue += 100.0f;
         break;
      }

      case 4: {
```

```
            cout << "Float token detected" << endl;
            break;
        }

        default: {
            cout << "Unknown token" << endl;
        }
    }

    cout << "Calculated Value: " << tokenValue << endl;

    return 0;
}
```

Dry Run:

Step	Operation	Value
1	tokenType = 2	2
2	multiplier = 5	5
3	tokenValue = 20 × 5	100
4	break statement executes	Exit switch

Expected Output:

Calculated Value: 100

Question 4: Reference Variable Initialization & Constant Binding Violations

1. Analyze the four reference operations and identify the errors.

Line 7:

int &refA;

Error: A reference variable must be initialized when it is declared. It cannot be declared without being bound to an existing variable.

Correct statement:

int &refA = a;

Line 11:

int &refLiteral = 100;

Error: A non-const lvalue reference cannot bind directly to an rvalue literal.

Correct statement:

const int &refLiteral = 100;

Line 14:

int &refInt = pi;

Error: The variable pi is a double, whereas refInt is an integer reference. A non-const reference cannot bind to a temporary created through an implicit type conversion.

Correct statement:

int piValue = 3;
int &refInt = piValue;

Line 18:

constRef = constRef + 10;

Error: A const reference cannot be used to modify the value it refers to.

Correct approach: Do not modify the const reference. Store the calculated result in another variable.

2. Explain lifetime extension of temporary objects.

When a temporary object is bound directly to a const reference, its lifetime is extended to match the lifetime of that reference.

For example:

const int &ref = 50;

Here, the temporary integer object containing 50 remains valid for the lifetime of ref.

However, the reference cannot be used to modify the temporary object.

3. Corrected C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int a = 25;

    int &refA = a;

    const int &refLiteral = 100;

    double pi = 3.14159;
    int piValue = pi;
    int &refInt = piValue;

    const int &constRef = 50;
    int result = constRef + 10;

    cout << a << " " << refA << endl;
    cout << refLiteral << endl;
    cout << refInt << endl;
    cout << result << endl;

    return 0;
}
```

4. Expected Console Output

```
25 25
100
3
60
```

Question 5: Dynamic Array Allocation & Stray Loop Semicolon

1. Explain the consequence of the semicolon at the end of the for statement.

The incorrect statement is:

for (int i = 0; i < size; ++i);

The semicolon at the end terminates the loop immediately, creating an empty loop body.

The following block:

```
{
   readings[i] = (i + 1) * 10;
}
```

executes separately after the loop.

Since i was declared inside the for loop, it is not accessible outside the loop. Therefore, using i in the following block causes a compilation error.

2. Identify the memory deallocation defect.

Incorrect statement:

delete readings;

The array was allocated using:

new int[size];

Therefore, it must be deallocated using:

delete[] readings;

Difference:

- delete is used to release memory allocated for a single object using new.
- delete[] is used to release memory allocated for an array using new[].

Using the wrong deallocation operator results in undefined behavior.

3. Corrected C++ Program

```
#include <iostream>
using namespace std;

int main() {
   int size = 5;

   int *readings = new int[size];

   for (int i = 0; i < size; ++i) {
```

```
        readings[i] = (i + 1) * 10;
    }

    int total = 0;

    for (int i = 0; i < size; ++i) {
        total += readings[i];
    }

    cout << "Total: " << total << endl;

    delete[] readings;

    return 0;
}
```

4. Dry Run and Final Output

Iteration	Array Value	Total
1	10	10
2	20	30
3	30	60
4	40	100
5	50	150

Total = 10 + 20 + 30 + 40 + 50 = 150.

Expected Output:

Total: 150

SECTION B – PSEUDO-CODE WRITING QUESTIONS

Question 1: Procedural-to-OOP Paradigm Transformation

1. Design Overview

- Create a class named WaterConsumer to manage consumer information.
- Declare all attributes as private to achieve data hiding and encapsulation.
- Use a constructor to initialize consumer details and validate meter readings.
- Calculate the units consumed by subtracting the previous reading from the current reading.
- Compute the bill using the given three-tier tariff system.
- Apply a 10% environmental surcharge to the entire bill.
- Display the final invoice with consumer details and bill amount.

2. Structured Pseudo-Code

```
CLASS WaterConsumer

PRIVATE:
   STRING consumerID
   REAL previousReading
   REAL currentReading
   REAL unitsConsumed
   REAL totalBill

PUBLIC:

   CONSTRUCTOR Initialize(id, prev, curr)

      IF prev < 0 OR curr < 0 OR curr < prev THEN
         DISPLAY "Invalid Meter Readings"
         totalBill = 0
         unitsConsumed = 0
      ELSE
         consumerID = id
         previousReading = prev
         currentReading = curr
```

```
        CALL CalculateConsumption()
    ENDIF

END CONSTRUCTOR

METHOD CalculateConsumption()

    unitsConsumed = currentReading - previousReading

END METHOD

METHOD ComputeBill()

    REAL bill = 0

    IF unitsConsumed <= 100 THEN
        bill = unitsConsumed * 1.50

    ELSE IF unitsConsumed <= 300 THEN
        bill = (100 * 1.50) +
               ((unitsConsumed - 100) * 2.50)

    ELSE
        bill = (100 * 1.50) +
               (200 * 2.50) +
               ((unitsConsumed - 300) * 4.00)

        bill = bill + (bill * 0.10)
    ENDIF

    totalBill = bill

END METHOD

METHOD DisplayBill()

    DISPLAY "Consumer ID: ", consumerID
    DISPLAY "Previous Reading: ", previousReading
    DISPLAY "Current Reading: ", currentReading
    DISPLAY "Units Consumed: ", unitsConsumed
    DISPLAY "Total Bill: ", totalBill
```

```
    END METHOD

ENDCLASS


BEGIN Main

    CREATE WaterConsumer object
    CALL Initialize("WC101", 1450.0, 1820.0)

    CALL ComputeBill()
    CALL DisplayBill()

END Main
```

3. Dry Run / Trace Table

Given:

- Previous Reading = 1450
- Current Reading = 1820

Units Consumed = 1820 − 1450 = 370 units.

Tariff Slab	Units	Rate	Amount
First 100 units	100	1.50	150
Next 200 units	200	2.50	500
Above 300 units	70	4.00	280
Subtotal	370		930
Environmental surcharge (10%)			93
Final Bill			1023

Expected Output:

Consumer ID: WC101
Previous Reading: 1450
Current Reading: 1820
Units Consumed: 370
Total Bill: 1023

Question 2: Multi-Tier Tax & Precision Cast Engine

1. Design Overview

- Create a class named TaxCalculationEngine.
- Accept gross annual salary and investment deductions as inputs.
- Apply the maximum deduction limits of 150,000 and 25,000.
- Calculate taxable income by subtracting eligible deductions from gross salary.
- Apply progressive tax rates based on the specified tax brackets.
- Add 4% Health and Education Cess to the raw tax.
- Convert the final tax from REAL to INTEGER using nearest-rupee rounding.

2. Structured Pseudo-Code

```
CLASS TaxCalculationEngine

PRIVATE:
   REAL grossAnnualSalary
   REAL deduction80C
   REAL deductionHealthInsurance
   REAL eligible80C
   REAL eligibleHealth
   REAL taxableIncome
   REAL rawTax
   REAL cess
   REAL finalTax

PUBLIC:

   METHOD CalculateTax(salary, d80C, dHealth)

      grossAnnualSalary = salary

      eligible80C = MIN(d80C, 150000)
      eligibleHealth = MIN(dHealth, 25000)

      taxableIncome = grossAnnualSalary -
```

```
            eligible80C -
            eligibleHealth

    IF taxableIncome < 0 THEN
        taxableIncome = 0
    ENDIF

    rawTax = 0

    IF taxableIncome <= 250000 THEN

        rawTax = 0

    ELSE IF taxableIncome <= 500000 THEN

        rawTax = (taxableIncome - 250000) * 0.05

    ELSE IF taxableIncome <= 1000000 THEN

        rawTax = (250000 * 0.05) +
                 ((taxableIncome - 500000) * 0.20)

    ELSE

        rawTax = (250000 * 0.05) +
                 (500000 * 0.20) +
                 ((taxableIncome - 1000000) * 0.30)

    ENDIF

    cess = rawTax * 0.04

    finalTax = rawTax + cess

    finalTax = INTEGER(finalTax + 0.5)

    DISPLAY "Taxable Income: ", taxableIncome
    DISPLAY "Raw Tax: ", rawTax
    DISPLAY "Health and Education Cess: ", cess
    DISPLAY "Final Tax: ", finalTax

END METHOD
```

ENDCLASS

BEGIN Main

REAL salary = 1200000
REAL d80C = 180000
REAL dHealth = 30000

CREATE TaxCalculationEngine object

CALL CalculateTax(salary, d80C, dHealth)

END Main

3. Dry Run / Trace Table

Given:

- Gross Annual Salary = 1,200,000
- Deduction 80C = 180,000
- Health Insurance Deduction = 30,000

Eligible deductions:

Particulars	Actual Amount	Maximum Limit	Eligible Amount
80C Deduction	180,000	150,000	150,000
Health Insurance	30,000	25,000	25,000

Taxable Income:

$$=1200000-150000-25000$$
$$=1025000$$

Tax Calculation:

Tax Slab	Taxable Amount	Rate	Tax

Up to 250,000	250,000	0%	0
250,001–500,000	250,000	5%	12,500
500,001–1,000,000	500,000	20%	100,000
Above 1,000,000	25,000	30%	7,500
Total Raw Tax			120,000

Health and Education Cess:

\[=120000\times\frac{4}{100}=4800 \]

Final Tax:

\[=120000+4800=124800 \]

After rounding = 124800.

Expected Output:

Taxable Income: 1025000
Raw Tax: 120000
Health and Education Cess: 4800
Final Tax: 124800

Question 3: Bitwise Mask Validation & Sensor Health Telemetry

1. Required Hexadecimal Bitmask Constants

Bit	Subsystem	Hexadecimal Mask
Bit 0	Gyroscope Locked	0x01
Bit 1	Accelerometer Calibrated	0x02

Bit 2	Barometric Altimeter Online	0x04
Bit 3	GPS Satellite Fix	0x08
Bit 4	Low Battery Warning	0x10
Bit 5	Over-Temperature Warning	0x20
Bit 6	Radio Telemetry Link	0x40
Bit 7	Fatal Actuator Failure	0x80

Combined masks:

- Mandatory Sensor Mask = 0x0F
- Inhibitor Mask = 0xB0
- Radio Link Mask = 0x40
- 8-bit Mask = 0xFF

2. Structured Pseudo-Code

```
CONSTANT GYROSCOPE = 0x01
CONSTANT ACCELEROMETER = 0x02
CONSTANT ALTIMETER = 0x04
CONSTANT GPS = 0x08

CONSTANT LOW_BATTERY = 0x10
CONSTANT OVER_TEMPERATURE = 0x20
CONSTANT RADIO_LINK = 0x40
CONSTANT FATAL_FAILURE = 0x80

CONSTANT MANDATORY_MASK = 0x0F
CONSTANT INHIBITOR_MASK = 0xB0
CONSTANT MASK_8BIT = 0xFF


METHOD CheckFlightReadiness(statusRegister)

    INTEGER missingMask
    INTEGER inhibitorFlags
```

```
INTEGER notStatus

inhibitorFlags = statusRegister AND INHIBITOR_MASK

notStatus = NOT statusRegister AND MASK_8BIT

missingMask = MANDATORY_MASK AND notStatus

DISPLAY "Status Register: ", statusRegister

IF (statusRegister AND LOW_BATTERY) != 0 THEN
   DISPLAY "Error: Low Battery Warning"
ENDIF

IF (statusRegister AND OVER_TEMPERATURE) != 0 THEN
   DISPLAY "Error: Over Temperature Warning"
ENDIF

IF (statusRegister AND FATAL_FAILURE) != 0 THEN
   DISPLAY "Error: Fatal Actuator Failure"
ENDIF

IF missingMask != 0 THEN

   DISPLAY "Missing Sensor Mask: ", missingMask

   IF (missingMask AND GYROSCOPE) != 0 THEN
      DISPLAY "Gyroscope Missing"
   ENDIF

   IF (missingMask AND ACCELEROMETER) != 0 THEN
      DISPLAY "Accelerometer Missing"
   ENDIF

   IF (missingMask AND ALTIMETER) != 0 THEN
      DISPLAY "Altimeter Missing"
   ENDIF

   IF (missingMask AND GPS) != 0 THEN
      DISPLAY "GPS Missing"
   ENDIF
```

```
    ENDIF

    IF inhibitorFlags == 0 AND missingMask == 0 THEN
        DISPLAY "Flight Ready"
    ELSE
        DISPLAY "Flight Not Ready"
    ENDIF

END METHOD
```

3. Trace Table for Test Byte 0x4F

Hexadecimal 0x4F = Binary 01001111.

Bit	Value	Status
Bit 0	1	Gyroscope available
Bit 1	1	Accelerometer available
Bit 2	1	Altimeter available
Bit 3	1	GPS available
Bit 4	0	No low battery warning
Bit 5	0	No temperature warning
Bit 6	1	Radio link active
Bit 7	0	No fatal failure

Calculations:

- Mandatory mask = 0x0F.
- Inhibitor flags = 0x4F AND 0xB0 = 0x00.
- Missing sensor mask = 0x00.
- All mandatory sensors are available.
- No inhibitor flags are active.

Output:

Status Register: 79
Flight Ready

Trace Table for Test Byte 0x3B

Hexadecimal 0x3B = Binary 00111011.

Bit	Value	Status
Bit 0	1	Gyroscope available
Bit 1	1	Accelerometer available
Bit 2	0	Altimeter missing
Bit 3	1	GPS available
Bit 4	1	Low battery warning
Bit 5	1	Over-temperature warning
Bit 6	0	Radio link inactive
Bit 7	0	No fatal failure

Calculations:

- Inhibitor flags = 0x3B AND 0xB0 = 0x30.
- Missing sensor mask = 0x04.
- XOR difference mask = 0x0F XOR 0x0B = 0x04.
- Altimeter sensor is missing.
- Low battery and over-temperature warnings are active.

Output:

Status Register: 59
Error: Low Battery Warning

Error: Over Temperature Warning
Missing Sensor Mask: 4
Altimeter Missing
Flight Not Ready

Question 4: Optimized Prime Factorization & Divisor Engine

1. Design Overview

- Accept a positive integer N from the user.
- Validate the input and reject zero or negative values.
- Handle the special case N = 1.
- Initialize the sum of proper divisors with 1.
- Check divisors from 2 up to the square root of N.
- Use the CONTINUE statement when a number does not divide N evenly.
- Add both the divisor and its paired divisor to the sum.
- Avoid double-counting when both divisors are equal.
- Classify the number as Perfect, Abundant, or Deficient.

2. Structured Pseudo-Code

```
BEGIN DivisorAndPrimeEngine

   INTEGER N
   INTEGER d
   INTEGER pairedDivisor
   INTEGER sumOfDivisors

INPUT_RETRY:

   DISPLAY "Enter a positive integer: "
   INPUT N

   IF N <= 0 THEN
      DISPLAY "Invalid Input. Enter a positive number."
      GOTO INPUT_RETRY
   ENDIF

   IF N == 1 THEN
      DISPLAY "Deficient Number"
      DISPLAY "Sum of Proper Divisors: 0"
```

```
    RETURN
  ENDIF

  sumOfDivisors = 1

  FOR d = 2 TO FLOOR(SQRT(N)) STEP 1

    IF N MOD d != 0 THEN
      CONTINUE
    ENDIF

    pairedDivisor = N / d

    sumOfDivisors = sumOfDivisors + d

    IF pairedDivisor != d THEN
      sumOfDivisors =
        sumOfDivisors + pairedDivisor
    ENDIF

  ENDFOR

  DISPLAY "Sum of Proper Divisors: ", sumOfDivisors

  IF sumOfDivisors == N THEN

    DISPLAY "Perfect Number"

  ELSE IF sumOfDivisors > N THEN

    DISPLAY "Abundant Number"

  ELSE

    DISPLAY "Deficient Number"

  ENDIF

END DivisorAndPrimeEngine
```

3. Dry Run for N = 28

Initial values:

- N = 28
- Sum of divisors = 1

Divisor (d)	N MOD d	Paired Divisor (N/d)	Sum of Divisors
Initial			1
2	0	14	17
3	1	Not calculated	17
4	0	7	28

Final proper divisors:

1, 2, 4, 7, 14

Sum = 1 + 2 + 4 + 7 + 14 = 28.

Since Sum = N, the number is a Perfect Number.

Output:

Enter a positive integer: 28
Sum of Proper Divisors: 28
Perfect Number

4. Dry Run for N = 12

Initial values:

- N = 12
- Sum of divisors = 1

Divisor (d)	N MOD d	Paired Divisor (N/d)	Sum of Divisors
Initial			1
2	0	6	9

3	0	4	16

Final proper divisors:

1, 2, 3, 4, 6

Sum = 1 + 2 + 3 + 4 + 6 = 16.

Since Sum is greater than N, the number is an Abundant Number.

Output:

Enter a positive integer: 12
Sum of Proper Divisors: 16
Abundant Number

Question 5: State-Machine ATM Controller with Sentinel Loops

1. Design Overview

- Create an ATM transaction controller using a menu-driven approach.
- Authenticate the cardholder using a four-digit PIN.
- Allow a maximum of three PIN attempts.
- Lock the card if all three attempts are incorrect.
- Display a menu for balance inquiry, deposit, withdrawal, and exit.
- Use a sentinel-controlled loop to continue transactions until the user selects Exit.
- Validate deposit amounts before adding them to the balance.
- Validate withdrawal amounts against the available balance.
- Ensure withdrawal amounts are exact multiples of 100 using modulus arithmetic.
- Use a switch-case statement to process the selected transaction.

2. Structured Pseudo-Code

BEGIN ATM_SessionController

```
INTEGER correctPIN = 4821
INTEGER enteredPIN
INTEGER attempts = 0
INTEGER choice

REAL balance = 1000
```

```
REAL amount

BOOLEAN authenticated = FALSE

WHILE attempts < 3 AND authenticated == FALSE

   DISPLAY "Enter 4-digit PIN: "
   INPUT enteredPIN

   IF enteredPIN == correctPIN THEN

      authenticated = TRUE
      DISPLAY "Authentication Successful"

   ELSE

      attempts = attempts + 1

      DISPLAY "Incorrect PIN"

   ENDIF

ENDWHILE

IF authenticated == FALSE THEN

   DISPLAY "Card Locked"
   RETURN

ENDIF


WHILE TRUE

   DISPLAY "----- ATM MENU -----"
   DISPLAY "1. Balance Inquiry"
   DISPLAY "2. Cash Deposit"
   DISPLAY "3. Cash Withdrawal"
   DISPLAY "4. Exit"

   DISPLAY "Enter your choice: "
   INPUT choice
```

```
SWITCH choice

    CASE 1:

        DISPLAY "Available Balance: ", balance
        BREAK

    CASE 2:

        DISPLAY "Enter Deposit Amount: "
        INPUT amount

        IF amount > 0 THEN

            balance = balance + amount

            DISPLAY "Deposit Successful"
            DISPLAY "Updated Balance: ", balance

        ELSE

            DISPLAY "Invalid Deposit Amount"

        ENDIF

        BREAK

    CASE 3:

        DISPLAY "Enter Withdrawal Amount: "
        INPUT amount

        IF amount <= 0 THEN

            DISPLAY "Invalid Amount"

        ELSE IF amount > balance THEN

            DISPLAY "Insufficient Balance"

        ELSE IF amount MOD 100 != 0 THEN
```

```
            DISPLAY "Amount Must Be Multiple of 100"

        ELSE

            balance = balance - amount

            DISPLAY "Withdrawal Successful"
            DISPLAY "Updated Balance: ", balance

        ENDIF

        BREAK

    CASE 4:

        DISPLAY "Thank You for Using ATM"
        RETURN

    DEFAULT:

        DISPLAY "Invalid Menu Selection"

  ENDSWITCH

 ENDWHILE

END ATM_SessionController
```

3. Dry Run / Test Trace

Initial values:

- Correct PIN = 4821
- Initial Balance = 1000
- Maximum PIN Attempts = 3

Authentication Phase:

Attempt	Entered PIN	Result

1	1234	Incorrect PIN
2	4821	Authentication Successful

The user is authenticated after the second attempt.

Transaction Phase:

Step	Transaction	Amount	Result	Balance
1	Initial Balance	—	—	1000
2	Withdrawal	250	Rejected (not a multiple of 100)	1000
3	Withdrawal	300	Successful	700
4	Exit	—	Session terminated	700

Explanation:

- The withdrawal of 250 is rejected because 250 MOD 100 is not equal to zero.
- The withdrawal of 300 is accepted because 300 is a multiple of 100 and is within the available balance.
- The updated balance becomes 1000 − 300 = 700.
- The user selects option 4 to terminate the session.

Expected Output:

Enter 4-digit PIN: 1234
Incorrect PIN

Enter 4-digit PIN: 4821
Authentication Successful

----- ATM MENU -----
1. Balance Inquiry
2. Cash Deposit
3. Cash Withdrawal
4. Exit

Enter your choice: 3
Enter Withdrawal Amount: 250
Amount Must Be Multiple of 100

----- ATM MENU -----
1. Balance Inquiry
2. Cash Deposit
3. Cash Withdrawal
4. Exit

Enter your choice: 3
Enter Withdrawal Amount: 300
Withdrawal Successful
Updated Balance: 700

----- ATM MENU -----
1. Balance Inquiry
2. Cash Deposit
3. Cash Withdrawal
4. Exit

Enter your choice: 4
Thank You for Using ATM